

Document Number: OAC-STD-001

Revision: I

UNIVERSAL COMMAND AND CONTROL (C2) INTERFACE (UCI)

UCI STANDARD



22 JANUARY 2026

Prepared By:
Open Architecture Collaborative Working Group (OACWG)

DISCLAIMER: The discussion of non-federal entities, methods, products, or services does not imply any endorsement by the United States Government, the Department of Defense, or the Department of the Air Force.

Distribution Statement A. Approved for public release: distribution is unlimited.

This page is intentionally left blank.

ABSTRACT

Universal Command and Control (C2) Interface (UCI) is a messaging standard supporting communication for mission-level C2. UCI provides a common message definition for performing mission operations and enables C2 coordination across weapon systems and weapon system elements such as sensors, vehicles, data products, and software. UCI is defined by the UCI Standard, which is comprised of the UCI Standard Document (i.e., this document), the UCI Schema, and the UCI Schema Style and Design Specification. This UCI Standard Document introduces UCI, introduces and references the UCI Schema (which defines UCI Messages) and UCI Schema Style and Design Specification (which provides rules (i.e., CERTs) and guidelines (i.e., GUIDs) for creating and using UCI Messages), defines UCI compliance, and documents how UCI is maintained. The UCI Standard Document does not prescribe the specific hardware, message encoding, or networking protocols as these technologies evolve rapidly and are heavily dependent upon specific program requirements. Additionally, there exists a companion document that addresses message integrity. It provides a message wrapper structure to enable the ability to follow a chain of trust for a message from one endpoint to another. This companion document is the UCI Secure Messaging Interface Definition Document (OAC-IDD-001).

This page is intentionally left blank.

Document Number: OAC-STD-001

Revision: I

REVISION RECORD

REVISION	DATE	DESCRIPTION AND SECTIONS AFFECTED
I	22 January 2026	Initial public release

See disclaimer and distribution A statements on cover page.

This page is intentionally left blank.

TABLE OF CONTENTS

1	Introduction.....	1
1.1	Overview and Purpose of UCI.....	1
1.2	Scope and Use of the standard.....	3
1.2.1	UCI Standard Document.....	5
1.2.2	UCI Schema.....	6
1.2.3	UCI Schema Style and Design Specification.....	7
1.2.4	UCI Normalized Interface Specification.....	7
1.2.5	Program Schema.....	8
2	UCI Schema Design.....	9
2.1	UCI Schema Data Format.....	9
2.2	UCI Schema Style and Design.....	9
3	UCI Compliance.....	10
4	Proposing Changes to the UCI Standard.....	12
5	UCI Versioning.....	13
5.1	UCI Standard Versioning.....	13
5.2	UCI Message Versioning.....	13
6	UCI Standard Governance.....	22
7	References.....	23
8	Acronyms and Abbreviations List.....	24
9	Glossary.....	25

This page is intentionally left blank.

LIST OF FIGURES

Figure 1.2-1 Elements of the UCI Standard.....	3
Figure 1.2-2 UCI Standard Versioning.....	4
Figure 1.2-3 Files and Structure of an Example UCI Release.....	5
Figure 1.2-4 Definition of UCI Schema and UCI Message.....	6
Figure 1.2-5 A UCI Message Set.....	7
Figure 1.2-6 Program Schema.....	8
Figure 5.2-1 UCI Message Versioning Designator Construction.....	18
Figure 5.2-2 Example of Versioning for a Structural Change.....	19
Figure 5.2-3 Example of Versioning for an Optional Change.....	20
Figure 5.2-4 UCI Message Versioning – Developer Quick Glance.....	21

This page is intentionally left blank.

LIST OF TABLES

Table 5.2-1 Structural Change Types.....	15
Table 5.2-2 Optional Change Types.....	16
Table 5.2-3 Non-Tracked Change Types.....	17
Table 7.0-1 Referenced Documents.....	23
Table 8.0-1 Acronyms.....	24
Table 9.0-1 Term Definitions.....	25

This page is intentionally left blank.

1 INTRODUCTION

1.1 Overview and Purpose of UCI

Universal Command and Control (C2) Interface (UCI) is a messaging standard supporting communication for mission-level C2. UCI defines a set of messages intended to standardize interfaces between software services for C2, and these messages have been defined and validated as fit for purpose across a variety of mission behaviors and heterogeneous mission system implementations.

The term “Command and Control” has a different meaning in different organizations. For the purposes of UCI, C2 is a type of information exchange necessary to provide an element of a System of Systems the ability to perform its mission and generate usable products. Included in this are messages that are specific to executing a function to create an effect (actions – commands/controls) and those that are needed in order to support the execution of the action (situational awareness or configurations/settings). In the context of UCI, both the action (command and control) messages and the supporting data messages are needed in order to perform the mission. Thus, the UCI Schema includes both command/control messages and situational awareness/data messages.

Programs adopting UCI have a wide variety of system requirements, constraints and planned implementation approaches. To support that diversity, and to reflect the fact that applicable technologies evolve rapidly, the UCI Standard does not prescribe hardware, operating systems, encodings, or network protocols. UCI does provide guidance on the qualities and constraints developers need to consider when determining the hardware requirements, network topology and deployment of software for interoperability in an operational system.

Additionally, a companion document to UCI defines a UCI message wrapper structure for addressing message integrity and supporting non-repudiation through forensic analysis that follows a chain of trust for a message from one endpoint to another across a variety of transports. This companion document is the UCI Secure Messaging Interface Definition Document (OAC-IDD-001).

UCI is maintained and released by the United States Air Force (USAF) with support from the Open Architecture Collaborative Working Group (OACWG).

The business objectives/goals of UCI with associated amplifying information follow:

- Engage in future function information technology/advanced technology environments
 - Prioritize changes that extend and modify the standard to enable new technologies to enhance warfighting capability
- Promote adaptability, flexibility, and expandability supporting weapons system design/goals
 - Deliver capability at the speed of relevance – come up with templates that help guide Adopting Programs (APs) with structure

- Expand UCI to meet the evolving needs of the DoD portfolio – the standards body is organized with industry Subject Matter Experts (SMEs) and community SMEs in order to provide solutions that support evolving needs of APs
- Support a variety of missions and domains
 - Provide support for multi-domain, open-ended mission objectives that systems have today
- Enable data exchanges across weapons systems
 - Provide a common message definition used across weapons systems
- Simplify integration
 - Promote common understanding and use of UCI Messages/Fields
 - Standardize suitable behaviors via Normalization
- Facilitate adoption of modern technologies
 - Supports/allows emerging formats and transports to be utilized without impacting UCI
 - Does not specify a format or transport/file format
- Balance complexity to increase speed of adoption
 - Balance between complexity and functionality
 - Increasing complexity increases the understanding needed to implement functions
 - Need functionality to meet evolving warfighter needs and capabilities
 - Limit multiple paths to accomplish same functionality
 - Increase UCI intuitiveness and reduce expertise needed to implement
- Reduce technical risk and overall cost of ownership of weapon system programs
 - Reduces technical risk by using a common message definition
- Enable affordable technology refresh and capability evolution
 - Enables technology refresh without impact because UCI is not technology dependent
 - Investigate backward compatibility potential – our processes consider backwards compatibility during the change process
- Enable reuse of system elements across weapon systems
 - Supports DoD acquisition strategy of acquiring capabilities deployable on multiple weapon systems
- Empower continuous-Authority To Operate (ATO) and Certificate to Field (CTF)
 - Work towards providing stable releases of UCI thus reducing the impact to fielded releases.

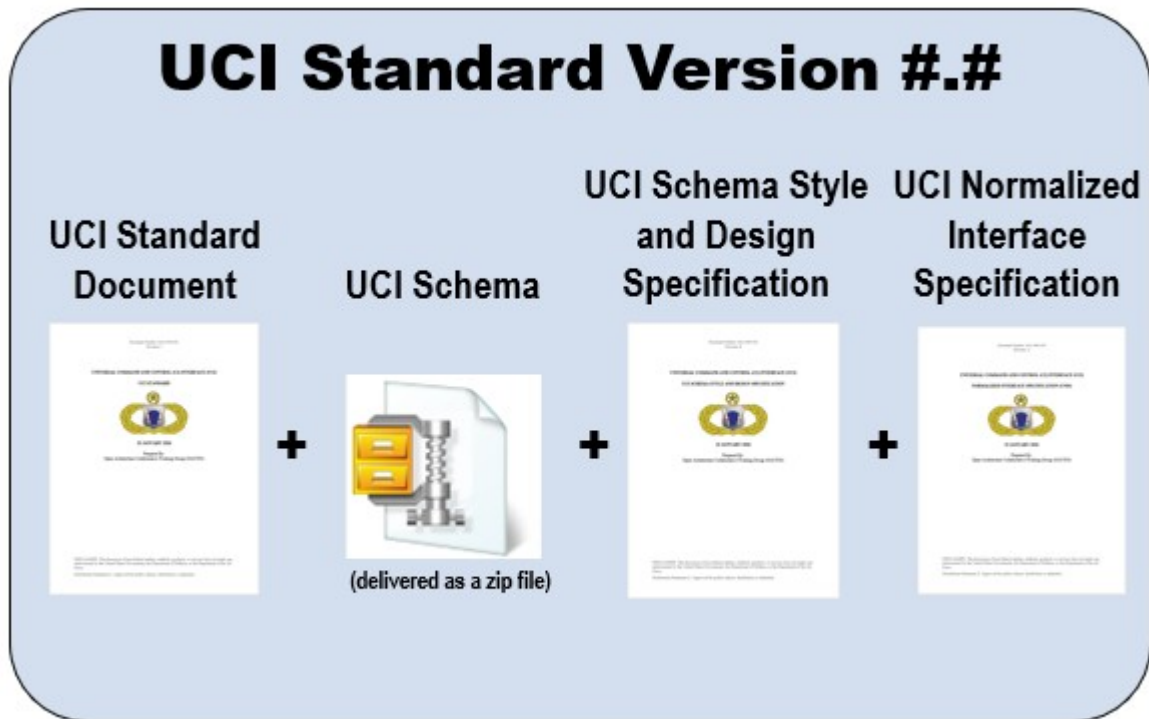
In short, the UCI Standard enables simultaneous, timely, and dynamic:

- Mission-level machine-to-machine control of multiple heterogeneous systems
- Mission planning for, and mission task execution by, multiple heterogeneous systems
- Reporting of plans and status from multiple heterogeneous systems.

1.2 Scope and Use of the standard

The UCI Standard defines each UCI release and is comprised of three elements as depicted in Figure 1.2-1 Elements of the UCI Standard:

- The UCI Standard Document (OAC-STD-001) (i.e., this document)
- The UCI Schema (OAC-STD-002), which is released as a zip file
- The UCI Schema Style and Design Specification (OAC-SPC-001)
- The UCI Normalized Interface Specification (OAC-SPC-002).



OAM-FIG-0104A

Figure 1.2-1 Elements of the UCI Standard

UCI releases are developed by the OACWG and published by the USAF in support of programs adopting UCI (i.e., Adopting Programs (APs)). The UCI Standard is used to define UCI releases, and Figure 1.2-2 UCI Standard Versioning, provides a depiction of the UCI Standard versioning approach. A version of the UCI Standard defining a UCI release is assigned a version number. The instances of a UCI Standard Document and UCI Schema are also labeled with the release number. Additionally, the version description document (VDD), UCI Standard Document, UCI Schema Style and Design Specification, and UCI Normalized Interface Specification also have the revision number of their respective document in their file names.

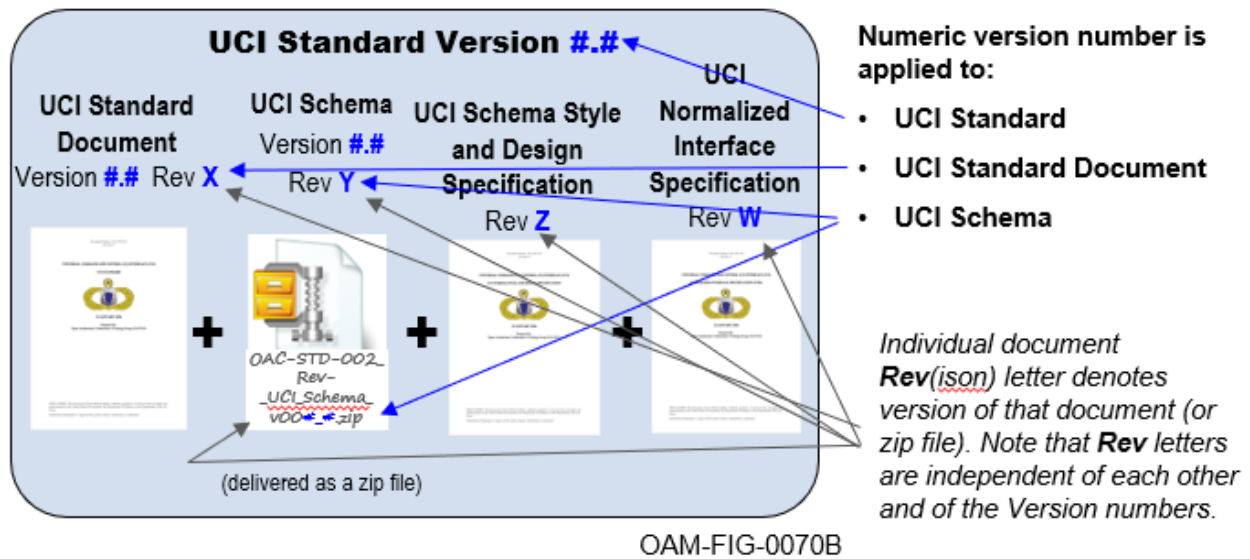


Figure 1.2-2 UCI Standard Versioning

Figure 1.2-3 Files and Structure of an Example UCI Release, provides an example a UCI release. UCI releases are assigned the version number of the UCI Standard that defines them, and the figure uses version 2.5 (i.e., 'UCI release v2.5') in its example.

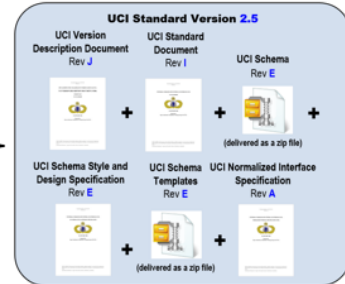
1.2.1 UCI Standard Document

This UCI Standard Document is the primary element of the UCI Standard, and references the other three elements as it provides:

- Requirements for UCI Message implementation (i.e., using UCI Messages), UCI Message definition (i.e., creating new and/or modifying existing messages)
- The definition of UCI compliance
- The definition of UCI primitive message pattern behaviors
- An explanation of UCI versioning
- A reference to the document describing the UCI change process.

A **'UCI release'** consist of six files. The example below depicts UCI release version 2.5:

01_OAC-VDD-001_RevJ_UCI_VersionDescriptionDocument_v2_5.docx
 02_OAC-STD-001_RevI_UCI_StandardDocument_v2_5.docx
 03_OAC-STD-002_RevE_UCI_Schema_v2_5.zip
 04_OAC-SPC-001_RevE_UCI_SchemaStyleAndDesignSpecification_v2_5.docx
 05_OAC-SPC-001_RevE_UCI_Templates_v2_5.zip
 06_OAC-SPC-002_RevA_UCI_NormalizedInterfaceSpecification_v2_5.docx



The **'UCI Schema'**, when unzipped, consists of these files:

Name

UCI_SecurityMarkings_v2_5_0.xsd
 UCI_MessageDefinitions_v2_5_0.xsd
 UCI_Versioning_v2_5_0.xsd
 README_DISCLAIMER.txt
 VDD.txt

These are schema definition files

These are supporting info files

Note – the UCI release version numbers (in this example version 2.5) are depicted in green text, and document revision numbers are depicted in brown text.

OAM-FIG-0062E

Figure 1.2-3 Files and Structure of an Example UCI Release

1.2.2 UCI Schema

The UCI Schema is a set of files that provide:

- Definition of UCI Message data types and security markings (provided in a programming-language agnostic manner using eXtensible Markup Language (XML) per the XML Schema Definition (XSD))
- UCI versioning and other release specific information.

The UCI Schema is released as a zip file, and an example of that zip file (as well as the other files in a UCI release) is depicted in Figure 1.2-3 Files and Structure of an Example UCI Release. In addition to XSD files, the UCI Schema includes files providing additional information:

- The ‘Schema Checker Results’ file is self-explanatory, and provides evidence that the UCI Schema meets its requirements (more on this topic in Section 3)
- The ‘VDD’ file provides version description document information about the changes in the current release of the UCI Schema.

The UCI Schema declares each UCI Message as a global element (Figure 1.2-4) in accordance with the UCI Schema Style and Design Specification.

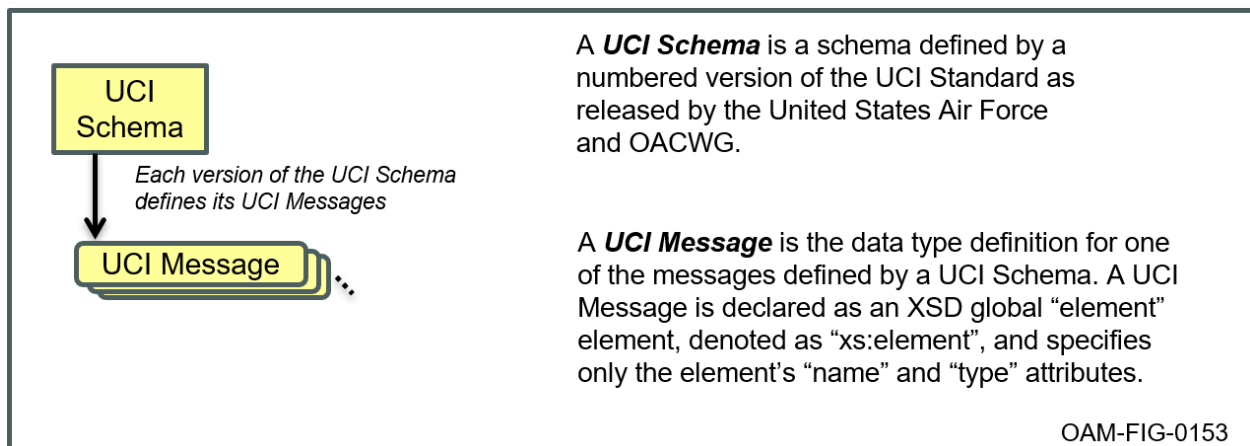


Figure 1.2-4 Definition of UCI Schema and UCI Message

To enable UCI adopters to use only the messages they need from a UCI Schema (versus a UCI Schema in its entirety), UCI defines a UCI Message Set as a subset (i.e., all or less) of a UCI Schema (Figure 1.2-5 A UCI Message Set).

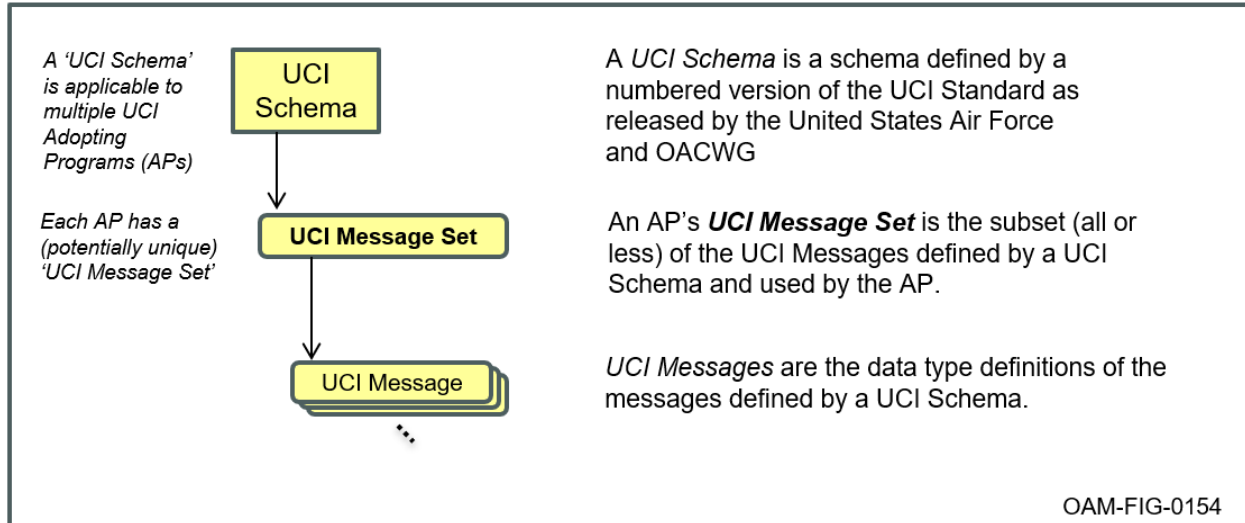


Figure 1.2-5 A UCI Message Set

1.2.3 UCI Schema Style and Design Specification

The third element of the UCI Standard (beyond the UCI Standard Document and the UCI Schema) is the UCI Schema Style and Design Specification (OAC-SPC-001). The UCI Schema Style and Design Specification provides guidance and requirements for developing and evolving the UCI Schema in the UCI Standard. This specification introduces the Certifications (CERTs) and Guidelines (GUIDs) that govern the representation of message content in the UCI Schema. CERTs are requirements that are strongly enforced where GUIDs/Guidelines while not strictly enforced are intended to provide guidance to developers to promote consistency across UCI Schema APs.

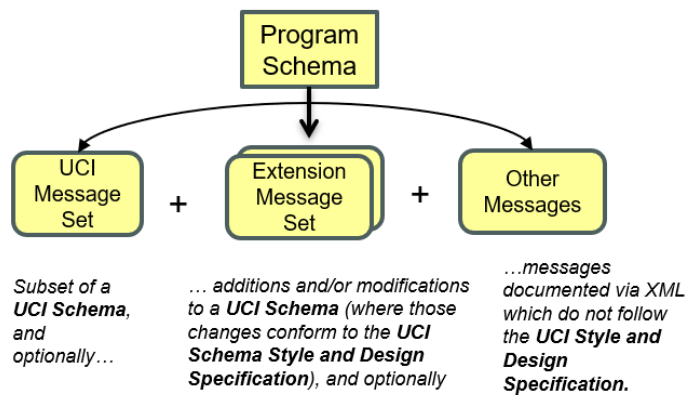
1.2.4 UCI Normalized Interface Specification

The fourth element of the UCI Standard is the UCI Normalized Interface Specification (OAC-SPC-002). This specification includes Certifications (CERTs) that govern UCI Message interaction patterns and behaviors.

1.2.5 Program Schema

The concept of a Program Schema (Figure 1.2-6) gives programs a significant degree of flexibility in how they can use UCI. A Program Schema can include:

- The AP's 'UCI Message Set' (i.e., UCI Messages from the AP's primary UCI Schema that are actually used by the AP), and/or
- One or more 'Extension Message Sets' representing planned or in-process UCI Schema changes (i.e., additional or modified messages) and/or changes to a UCI Schema that are program-specific and are not intended to be added to a future version of the UCI Schema. By definition, an Extension Message Set must conform to a UCI Schema Style and Design Specification, and/or
- 'Other Messages' documented via XSD that are independent of UCI (i.e., do not conform to a UCI Schema Style and Design Specification).



A **Program Schema** lets programs combine three types of message sets with different levels of alignment with UCI.

A **Program Schema** is composed of a **UCI Message Set**, one or more optional **Extension Message Sets**, and optionally Other Messages.

This flexible approach lets each program tailor how it uses UCI, and how closely it is aligned to the UCI Standard.

OAM-FIG-0155

Figure 1.2-6 Program Schema

2 UCI SCHEMA DESIGN

2.1 UCI Schema Data Format

The UCI Standard documents its message format using an XML 1.0 Schema Definition (XSD). Rigorous definition of World Wide Web Consortium (W3C) XML Schema Definition Language (XSD) 1.0 (Part 1: Structures and Part 2: Datatypes) upon which the UCI Messages are based is referenced in Table 7.0-1, Referenced Documents.

2.2 UCI Schema Style and Design

In addition to the W3C XSD specification, the UCI Standard specifies design and naming conventions for the definition of the UCI Schema. These conventions are meant to profile allowable XSD constructs, as well as provide structure and field definitions to help ensure elements in the schema are consistent and reusable. The UCI Schema Style and Design Specification specifies principles and enforceable rules (i.e., CERTs) for UCI Schema data components.

3 UCI COMPLIANCE

UCI compliance by an AP is accomplished by adherence to RQMT USTD-000120, RQMT USTD-000629, and RQMT USTD-000436.

RQMT USTD-000120 [A UCI Component shall use messages defined in the UCI Schema for interactions with other UCI Components.]

Evidence of meeting this requirement would include test results that show the UCI Component sends and/or receives the UCI Messages it uses. A UCI Component is anything that sends and/or receives UCI Messages.

UCI compliance excludes non-messaging interactions, prescription of wire formats and protocols, and a distinction between types of components.

UCI Messages are restricted by their format as defined in the UCI Schema. There is primitive behavior defined in the UCI Normalized Interface Specification (UNIS) that is also required. The annotations in the UCI Schema are for guidance.

RQMT USTD-000629 [UCI Message interactions shall conform to the CERTs in the UCI Normalized Interface Specification.]

Evidence of meeting this requirement would include test results that show the UCI Components sends and/or receives UCI Messages in accordance with the role that the UCI Component plays in the interaction.

An additional requirement applies to the UCI Schema itself, versus applying to adopters of the UCI Schema.

RQMT USTD-000125 [A UCI Schema shall conform to the CERTs in the UCI Schema Style and Design Specification.]

Conformance primarily involves the OACWG and OAM ensuring that each released version of the UCI Schema meets this requirement, and secondarily involves all parties involved with requesting or developing changes to the UCI Schema ensure that such changes align with the UCI Schema Style and Design Specification.

To meet this requirement, the OAM program will execute the processes established in the OACWG Governance Plan (OMSC-PGM-0031) for performing updates to the UCI Schema. The processes include, but are not limited to, conducting reviews of emerging changes and using the UCI SchemaChecker tool to programmatically enforce the UCI Schema Style and Design Specification CERTs.

In addition, in order to fully embrace supporting the integration of multiple adopting programs or users, UCI utilizes unique identifiers and specifies that they conform to an industry standard. The Universally Unique Identifier (UUID) elements in the schema are used to ensure that there is a low probability of collision (duplication). This is important when multiple systems share UCI data. Thus, UCI requires that identifiers within the schema conform to any version of the Leach-Salz UUID variant (variant 1) or the nil UUID described in Internet Engineering Task Force (IETF) Request for Comments (RFC) 4122, which specifies the format and generation constraints to reduce the potential for collision when creating UUIDs.

RQMT USTD-000436 [UCI UUIDs shall conform to any version of the Leach-Salz UUID variant (variant 1) or the nil UUID described in RFC 4122.]

RFC 4122 provides multiple ways to create UUIDs, including two mechanisms that result in deterministic behavior. In other words, given a known set of inputs, the same UUID can be created and recreated. This supports use cases where there is need for a UUID to be recreated to the same value. The adopting program/user is responsible for proving conformance to RFC 4122.

To standardize UUID representation in UCI, align with industry best practices related to UUID representation in text-based formats or user-facing applications, and to improve human-readability of UCI messages, UCI UUIDs shall conform to the UUID string representation described in RFC-4122 (i.e. xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx where “x” is a hexadecimal digit).

RQMT USTD-000673 [UCI UUIDs shall conform to the UUID string representation described in the “Namespace Registration Template” section of RFC-4122.]

4 PROPOSING CHANGES TO THE UCI STANDARD

As programs adopt the UCI Standard, they may find that one or more elements of the UCI Standard (i.e., UCI Standard Document, UCI Schema, and/or UCI Schema Style and Design Specification) appear insufficient to convey the information they require. The first step is to work with UCI experts/authorities to clearly understand the elements of the Standard and how they can and should be used. In many cases, this collaboration can clear up potential issues. In its Program Schema, a program may augment the UCI Message Set used from a UCI Schema with one or more Extension Message Sets and/or Other Messages. Depending on the needs of the program, this approach can offer an interim or permanent solution to the program's messaging needs.

If it is determined that a deficiency in the UCI Standard does exist, the program will need to modify the applicable UCI Standard element(s) in accordance with the process and methods defined in the OACWG Governance Plan. The OACWG Governance Plan defines the process and schedule to be followed in proposing and achieving an approved change to the UCI Standard. Prior to generating a request for approval of a change, the adopting program will need to engineer a viable change to the UCI Standard that reflects their specific need.

5 UCI VERSIONING

5.1 UCI Standard Versioning

The UCI approach to versioning is to assign release numbers using 2 integers separated by a decimal point (i.e., X.Y) where the X is increased as determined by the UCI authorities based on their belief that the release is a significant change, and the Y is either set to zero (when the X is incremented) or incremented (for non-significant releases). The release number is not indicative of impact to an individual program nor backwards compatibility with a previous release. Each UCI release has the UCI Standard X.Y (i.e., Overall, UCI Standard Document X.Y, UCI Schema X.Y, and UCI Schema Style and Design Specification X.Y release) and a Version Description Document (VDD).

Each AP must analyze the impact of changes to the Standard based upon their own implementation and determine the appropriate timing for moving to a new version of the Standard release. The UCI Schema includes versioning information for each message, complexType, and simpleType associated with a given release. For specific version references, see the attribute “uci:version” for each message, complexType, and simpleType in the corresponding schema file. More versioning details are described in Section 5.2, UCI Message Versioning.

Each release of the UCI Standard is accompanied by a VDD that records the overall version of the Standard and the revision letter of each constituent document. Each document has a revision letter that is incremented each time that document incorporates changes and includes an associated change history table that describes the scope of the change.

5.2 UCI Message Versioning

The purpose of the UCI Message versioning is twofold: it provides program developers with the ability to identify the level of impact of changes to messages between two different versions of UCI Schema and identify categories of changes to consistently quantize the impact. This level of detail allows developers to determine the effort necessary to update their program to use the new UCI Schema release. UCI Message Versioning provides a version identifier for each Message in the UCI Schema. In support of this message versioning, the process also tracks, at the same level of detail, changes to complexTypes and simpleTypes specifying the type of category and traceability occurring within engineering versions and official releases of the Schema. Note: Messages in UCI are constructed from a complexType with name of message appended with MT (<MsgName>MT). The version number assigned to Messages will be directly inherited from its associated MT complexType.

UCI Message versioning consists of two categories of changes: Structural and Optional, and two levels of traceability: Direct and Indirect. Definitions for each change category type and levels of traceability are as follows.

Change Category:

- **Structural:** A change that is not backward compatible, i.e., code is impacted. Table 5.2-1, Structural Change Types, lists the type of changes that can result in impacts. Note: “Can” refers to assessment assuming worst case scenario on how that change can be realized at higher levels of traceability.
- **Optional:** A change that is backward compatible, i.e., developer to be cognizant of change. Table 5.2-2, Optional Change Types, lists the type of changes that result in no impact. Note: These types of changes have no impact when the change is traversed to higher levels of traceability.

Traceability Level:

Direct: A change to:

- a simpleType (e.g., modify pattern restriction, add an enum value)
- a complexType element (e.g., adding, deleting, modifying an element).

Indirect: A change to:

- a complexType element that uses a modified simpleType
- a complexType that extends from a modified base complexType
- a simpleType that is derived from a modified simpleType
- an inherited change to an element that is from a modified direct or indirect complexType or simpleType.

UCI Message versioning has also identified categories of changes not tracked for versioning purposes. These non-tracked changes are clarified as follows:

- **Non-Tracked:** A change that can occur but is deemed as not impactful to developer at this time and not tracked for versioning purposes. Table 5.2-3, Non-Tracked Change Types, lists the type of changes that are not tracked for versioning purposes.

Table 5.2-1 Structural Change Types

Structural Change Type	Amplifying Description
ComplexTypes	
Modify multiplicity from list to single element	Modifying an existing element from a list to a single element
Modify multiplicity from single element to list	Modifying an existing single element to a list
Decreasing multiplicity of list	Decreasing the value of maxOccurs for existing element
Delete element from complexType	Deleting an element from a sequence or choice complexType
Add required element to a complexType	Adding a new required element to a sequence complexType
Modify type used in existing element	Modify type used in an existing element to a different type (new or existing)
Modify element from optional to required	Modifying an existing optional element to required
Modify element from required to optional	Modifying existing required element to optional
Delete existing PET	Delete an existing Polymorphic Extension Type (PET)
Modify complexType from xs:choice to xs:sequence	Modifying the compositor type of a complexType from choice to sequence.
Modify complexType from xs:sequence to xs:choice	Modifying the compositor type of a complexType from sequence to choice.
Modify complexType to extend from type (includes abstract).	Modifying the derivation method of a complexType to extend from another complexType (including abstract).
Modify complexType used to extend by an existing type (includes abstract).	Modifying the derivation method of an existing complexType to extend using a different complexType (including abstract).
Modify complexType to no longer extend from type (includes abstract).	Modifying the derivation method of an existing complexType to no longer extend from another complexType (including abstract).
SimpleTypes	
Delete enum value	Deleting one or more values in an existing simpleType enum
Modify simpleType restriction facets	Modifying the restriction of facets for an existing simpleType: length, minLength, maxLength, pattern, minInclusive, maxInclusive, minExclusive, maxExclusive
Modify simpleType restriction base type	Modifying restriction base type from one type (xs primitive or simpleType) to another type (xs:primitive or simpleType)
Other	

Table 5.2-2 Optional Change Types

Optional Change Type	Amplifying Description
complexType	
Add optional element	Adding new optional element to a sequence complexType
Add element to a choice type	Adding new option to a choice complexType
Increasing multiplicity of list	Increasing the value of maxOccurs for an existing element
simpleTypes	
Adding new enum values to simpleTypes	Adding to simpleType enum one or more new values
Other	
Delete complexType/simpleType/Message	Deleting existing complexType/simpleType/Message (allows for documenting change) If message(s), covered by deprecation process and Notice of Significant Change

Table 5.2-3 Non-Tracked Change Types

Non-Tracked Change Type	Amplifying Description
ComplexTypes	
Reorder elements in complexType	Change to order of elements in a sequence complexType. Order of elements do not impact code.
Adding new complexType	Adding new complexType. Change impact is applied where it is first used
Rename complexType/Message	Renaming an existing complexType/Message. Treated as an Add/Delete in versioning.
Rename element	Renaming an existing element in a complexType. Treated as an Add/Delete in versioning.
SimpleTypes	
Reorder enum values	Changes to order of existing enum values to simpleType enums. Order of enum values should not impact code.
Insertion of new enum values not at the end	Order of enum values not impacted if new values inserted other than at end. Location of enum values should not impact code.
Adding new simpleType	Adding new simpleType. Change impact is applied where it is first used.
Rename simpleType	Renaming an existing simpleType. Treated as an Add/Delete in versioning.
Rename enum value	Renaming one or more values in an existing simpleType enum. Treated as an Add/Delete in versioning.
Other	
Change to annotation	Any detected change to annotations for messages, simpleTypes, complexTypes will not be tracked for versioning impacts.
Modify name of existing imported/included file	Modify name of existing imported/included file. This will be tracked at the schema versioning level.
Exclude xml schema namespace changes	Updating “xml” schema namespace has global impacts to all messages and types. Updating “xml” schema” namespace will be handled as a special case. Notification will be assessed on the content of the change and could be documented in either Release Notes or an appropriate Notice of Significant Change (NOSC).

The UCI Message version designator (uci:version) is then composed using the two category of changes and two levels of traceability resulting in four alpha-numeric components: Indirect Structural, Direct Structural, Indirect Optional, and Direct Optional, as shown in Figure 5.2-1, UCI Message Versioning Designator Construction, as component position A, B, C and D, respectively. The components in the UCI versioning schema are independent – only the component associated with the category/traceability of change is updated, all other components remain the same.

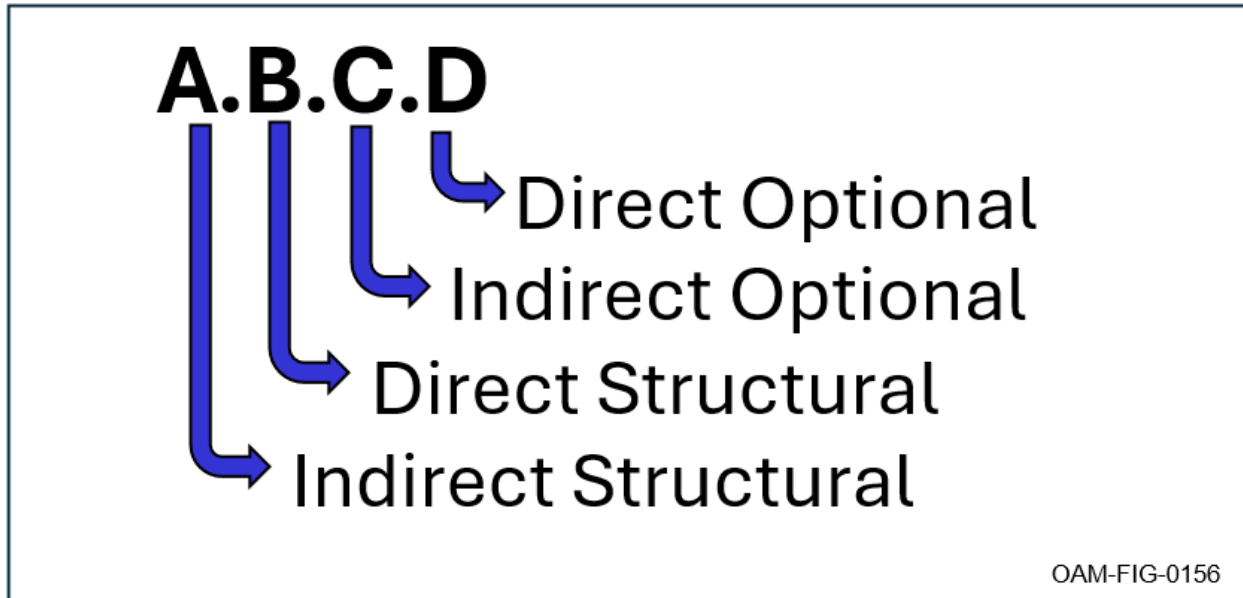


Figure 5.2-1 UCI Message Versioning Designator Construction

The following figures provide simple examples of how structural and optional change categories are applied through the traceability level from the change: direct or indirect.

Example of Structural Change: complexTypeE to MessageH

Current:

complexTypeE uci:version = 000.000.000.000

complexTypeF uci:version = 000.000.000.000

complexTypeG uci:version = 000.000.000.000

messageH uci:version = 000.000.000.000

- complexTypeE experiences a **structural** change (new required element)
 - complexTypeF uses complexTypeE
 - complexTypeG uses complexTypeF
 - message is created from complexTypeG

UCI Versioning Results:

- complexTypeE has a direct **structural** change due to addition of a new required element
 - complexTypeF has an indirect **structural** change due to direct structural change in complexTypeE
 - complexTypeG has an indirect **structural** change due to indirect **structural** change in complexTypeF
 - message has an indirect structural change due to indirect **structural** change in complexTypeG

Version Change:

complexTypeE uci:version = 000.001.000.000

complexTypeF uci:version = 001.000.000.000

complexTypeG uci:version = 001.000.000.000

message uci:version = 001.000.000.000 ← message inherits version from used complexTypeG

OAM-FIG-0157

Figure 5.2-2 Example of Versioning for a Structural Change

Example of Optional Change: simpleTypeW to MessageZ***Current:****simpleTypeW uci:version = 000.000.000.000**complexTypeX uci:version = 000.000.000.000**complexTypeY uci:version = 000.000.000.000/**complexTypeZ uci:version = 000.000.000.000**messageZ uci:version = 000.000.000.000*

- **simpleTypeW** experiences an **optional** change (new enum value)
 - **complexTypeX** uses **simpleTypeW**
 - **complexTypeY** uses **complexTypeX**
 - **complexTypeZ** uses **complexTypeY**
 - **messageZ** is created from **complexTypeZ**

UCI Versioning Results:

- **simpleTypeW** has a **direct optional** change due to addition of a new enum value
 - **complexTypeX** has an **indirect optional** change due to **direct optional** change to **simpleTypeW**
 - **complexTypeY** has an **indirect optional** change due to **direct optional** change in **complexTypeX**
 - **complexTypeZ** has an **indirect optional** change due to **indirect optional** change in **complexTypeY**
 - **messageZ** has an **indirect optional** change due to **indirect optional** change in **complexTypeZ**

Version Change:*simpleTypeW uci:version = 000.000.000.001**complexTypeX uci:version = 000.000.001.000**complexTypeY uci:version = 000.000.001.000**complexTypeZ uci:version = 000.000.001.000**messageZ uci:version = 000.000.001.000 ←message inherits version from complexTypeZ*

OAM-FIG-0158

Figure 5.2-3 Example of Versioning for an Optional Change

The components in the UCI Message versioning designator are ordered based on the expected level of effort (LOE) for the developer to identify the change and incorporate the change into their program (see Figure 5.2-4, UCI Message Versioning – Developer Quick Glance). For a formal release, the numerical component associated with the change category/traceability is incremented by one – example, MDT version 001.001.001.001 to MDT version 001.001.002.001 indicates an indirect optional changed occurred in the formal release.

Figure 5.2-4, UCI Message Versioning – Developer Quick Glance, provides a quick glance on the expected impact for each change category and traceability.

Structural	Recode: Yes Developer: Needs to Investigate LOE: Medium	Recode: Yes Developer: Needs to Investigate LOE: High
	Recode: No Developer: SA LOE: Low	Recode: No Developer: SA LOE: Medium
Optional	Direct	Indirect

OAM-FIG-0159

Figure 5.2-4 UCI Message Versioning – Developer Quick Glance

For clarification, the following information is provided to further define the work level associated with each change category and traceability;

- **Recode** – Identifies if the change impacts the programs code. Options include:
 - Yes: code is impacted, not backward compatible
 - No: code is not impacted, backward compatible
- **Developer** – Identifies the task the developer needs to perform. Options include:
 - Needs to Investigate: identify the change and incorporate into program
 - SA (Situational Awareness): be cognizant of change
- **LOE** – Identifies the anticipated LOE associated with incorporating change into program, if applicable.

6 UCI STANDARD GOVERNANCE

The UCI Standard is managed and sustained by the Open Architecture Management (OAM) program and governed by the process as defined in OMSC-PGM-0031 OACWG Governance Plan.

7 REFERENCES

The documents listed in Table 7.0-1 Referenced Documents, are cited throughout this document. For dated references, only the revision cited applies. For undated references, the latest revision of the referenced document (including any addendums or corrigenda) applies.

Table 7.0-1 Referenced Documents

Document Number	Document Title	Revision	Date
OMSC-PGM-031	OACWG Governance Plan	J	05 June 2025
OAC-STD-002	Universal Command and Control (C2) Interface (UCI) Schema	E	22 January 2026
OAC-SPC-001	Universal Command and Control (C2) Interface (UCI) Schema Style and Design Specification	E	22 January 2026
OAC-SPC-002	Universal Command and Control (C2) Interface (UCI) Normalized Interface Specification (UNIS)	A	22 January 2026
OAC-IDD-001	Universal Command and Control (C2) Interface (UCI) Secure Messaging Interface Definition Document (IDD)	-	22 January 2026
OAC-VDD-001	UCI Standard Version Description Document (VDD)	J	22 January 2026
N/A	Extensible Markup Language (XML) 1.0 (Fifth Edition)	1.0	26 November 2008
N/A	XML Schema Part 1: Structures Second Edition	1.0	28 October 2004
N/A	XML Schema Part 2: Datatypes Second Edition	1.0	28 October 2004
IETF RFC 4122	A Universally Unique IDentifier (UUID) URN Namespace		July 2005

8 ACRONYMS AND ABBREVIATIONS LIST

Table 8.0-1 Acronyms

Acronym	Definition
AP	Adopting Program
C2	Command and Control
CERT	Certification Requirement
GUID	Guidance
IETF	Internet Engineering Task Force
LOE	Level of Effort
MDT	Message Data Type
MT	Message Type
NOSC	Notice of Significant Change
OACWG	Open Architecture Collaborative Working Group
OAM	Open Architecture Management
OMS	Open Mission Systems
OMSC	OMS CDRL (Contract Deliverable Requirement List)
PET	Polymorphic Extension Type
PGM	Program
RFC	Request for Comments
SA	Situation Awareness
SME	Subject Matter Expert
UCI	Universal C2 Interface
USAF	United States Air Force
W3C	World Wide Web Consortium
XML	eXtensible Markup Language
XSD	XML Schema Definition

9 GLOSSARY

Table 9.0-1 Term Definitions

Term	Definition
Extension Message Set	In the context of UCI, Extension Message Set refers to a set of messages which are not a UCI Message Set but which conform to the UCI Schema Style and Design Specification. The messages may be additions or modifications to a UCI Schema which are planned as updates for a future UCI Schema version, or the messages may be program unique messages not intended to be added to a future UCI Schema version.
Other Messages	In the context of UCI, Other Messages refers to Program Schema messages documented via XML but which do not conform to the UCI Schema Style and Design Specification.
Program Schema	In the context of UCI, Program Schema refers to a set of messages used by a program (or programs) where the messages are documented using XML and include: (Optionally) A UCI Message Set (Optionally) One or more Extension Message Sets (Optionally) Other Messages.
UCI Message	A UCI Message is a data type definition for one of the messages defined by a UCI Schema. A UCI Message is declared as an XSD global “element” element, denoted as “xs:element”, and specifies only the element’s “name” and “type” attributes.
UCI Message Set	An AP’s UCI Message Set is the subset (all or less) of the UCI Messages defined by a particular UCI Schema and used by the AP.
UCI Schema	A UCI Schema is a schema defined by a numbered version of the UCI Standard as released by the USAF and OACWG. A UCI Schema defines UCI Messages via a set of C2 message type definitions defined using XML with each message declared as a global XML Schema Definition (XSD) element.